

TP SNT – Mini-jeu Python : Pierre–Feuille–Ciseaux

Niveau : Seconde | Durée : 1 heure | Outils : Jupyter Notebook, Python 3

0. Objectifs

- Prendre en main Jupyter (exécuter, modifier une cellule).
- Manipuler : variables, listes, entrée clavier, conditions, boucle.
- Programmer un mini-jeu jouable, puis l'améliorer par étapes.

1. Cours – Bases utiles

1.1 Variables et types

Une **variable** nomme et stocke une valeur.

```
# entier (int) et chaîne (str)
score = 0
nom = "Alice"
```

1.2 Entrée clavier

`input("message")` lit une chaîne saisie par l'utilisateur.

```
j = input("Ton choix : ") # renvoie une str
```

1.3 Listes et appartenance

```
options = ["pierre", "feuille", "ciseaux"]
if "pierre" in options:
    print("ok")
```

1.4 Hasard

```
import random  
o = random.choice(options) # choix aléatoire dans la liste
```

1.5 Conditions

```
if j == o:  
    print("Égalité")  
elif (j, o) in [("pierre", "ciseaux"), ("feuille", "pierre"), ("ciseaux", "feuille")]:  
    print("Victoire")  
else:  
    print("Défaite")
```

1.6 Boucles

```
for i in range(5): # répète 5 fois  
    ...  
# while répète tant que la condition est vraie  
while score_j < 3 and score_o < 3:  
    ...
```

2. Programme de base – à recopier

Copiez ce programme (≈15 lignes) dans Jupyter puis exécutez-le.

```
import random # hasard pour le choix de l'ordinateur

options = ["pierre", "feuille", "ciseaux"]
score_j, score_o = 0, 0 # scores joueur / ordinateur

for manche in range(5): # 5 manches fixes
    j = input("Choisis pierre, feuille ou ciseaux : ").strip().lower()
    if j not in options:
        print("Entrée invalide, manche ignorée.\n")
        continue

    o = random.choice(options)
    print("Ordinateur :", o)

    if j == o:
        print("Égalité.\n")
    elif (j, o) in [("pierre", "ciseaux"), ("feuille", "pierre"), ("ciseaux", "feuille")]:
        score_j += 1; print("Tu gagnes la manche.\n")
    else:
        score_o += 1; print("L'ordinateur gagne la manche.\n")

print("Score final :", score_j, "-", score_o)
```

3. Améliorations progressives

3.1 Saisie robuste (à recopier *d'abord*)

But : redemander tant que l'entrée n'est pas valide.

Remplacez uniquement la partie « lecture de j » dans la boucle.

```
j = input("Choisis pierre, feuille ou ciseaux : ").strip().lower()
while j not in options:
    j = input("Saisie invalide. Réessaye (pierre/feuille/ciseaux) : ").strip().lower()
```

Question : Pourquoi `while j not in options:` garantit-il qu'on sort avec une saisie correcte ?

3.2 Gagnant à 3 points avec while (à recopier ensuite)

But : on joue jusqu'à ce que l'un atteigne 3 points (nombre de manches variable).

```
import random

options = ["pierre", "feuille", "ciseaux"]
score_j, score_o = 0, 0

# Boucle tant que personne n'a 3 points
while score_j < 3 and score_o < 3:
    # saisie robuste
    j = input("Choisis pierre, feuille ou ciseaux : ").strip().lower()
    while j not in options:
        j = input("Saisie invalide. Réessaye (pierre/feuille/ciseaux) : ").strip().lower()

    # coup de l'ordinateur
    o = random.choice(options)
    print("Ordinateur :", o)

    # résultat de la manche
    if j == o:
        print("Égalité.\n")
    elif (j, o) in [("pierre", "ciseaux"), ("feuille", "pierre"), ("ciseaux", "feuille")]:
        score_j += 1; print("Tu gagnes la manche.\n")
    else:
        score_o += 1; print("L'ordinateur gagne la manche.\n")

print("Score final :", score_j, "-", score_o)
```

Question : En passant de `for range(5)` à `while score_j < 3 and score_o < 3`, que devient le nombre de manches ? Justifie.

3.3 Décompte des égalités (à l'élève de trouver)

- Créer `egalites = 0` avant la boucle.
- *Incrémenter* uniquement dans le cas `if j == o:`.
- Afficher à la fin : `print("Égalités :", egalites)`.

Question : Pourquoi ne faut-il pas incrémenter `egalites` après la boucle ?

3.4 Extension « Lézard–Spock » (à l'élève de trouver)

But : passer à 5 choix : ["pierre", "feuille", "ciseaux", "lézard", "spock"].

Rappels des règles gagnantes (phrases en français) :

- **Pierre** écrase **Ciseaux**
- **Pierre** écrase **Lézard**
- **Feuille** couvre **Pierre**
- **Feuille** réfute **Spock**
- **Ciseaux** coupent **Feuille**
- **Ciseaux** décapitent **Lézard**
- **Lézard** mange **Feuille**
- **Lézard** empoisonne **Spock**
- **Spock** brise **Ciseaux**
- **Spock** vaporise **Pierre**

Consignes :

1. Étendre options à 5 éléments et adapter les messages de saisie.
2. Représenter les règles avec un *ensemble* de paires gagnantes (ou un dictionnaire).

Défi bonus : afficher une phrase explicative (ex. « Spock vaporise pierre ») via un dictionnaire `explication[(j,o)]`.

4. Mémo

- **for** : nombre d'itérations *fixe* • **while** : s'arrête quand la *condition* devient fausse.
- **Saisie robuste** : boucler tant que l'entrée n'appartient pas à options.
- **Compteur** : initialiser avant la boucle, incrémenter au bon endroit.
- **Hasard** : `random.choice(liste)`.
- **Test victoire** : appartenance à un ensemble de paires gagnantes.

```
# Exemple d'égalité comptée
egalites = 0
if j == 0:
    egalites += 1
    print("Égalité")
```

5. QCM (20 questions, 1 bonne réponse)

1. Que renvoie `input()` ?

- A. Un entier
- B. Un flottant
- C. Une chaîne de caractères
- D. Un booléen

2. Quel module faut-il pour `choice` ?

- A. `math`
- B. `random`
- C. `sys`
- D. `time`

3. Quelle écriture crée une liste valide ?

- A. `("pierre", "feuille", "ciseaux")`
- B. `{"pierre", "feuille", "ciseaux"}`
- C. `["pierre", "feuille", "ciseaux"]`
- D. `<"pierre", "feuille", "ciseaux">`

4. Que fait `strip()` ?

- A. Met en minuscules
- B. Enlève espaces/retours autour
- C. Trie les lettres
- D. Remplace les espaces par `_`

5. `j not in options` est vrai si :

- A. `j` est dans `options`
- B. `j` n'est pas dans `options`
- C. `options` est vide
- D. `j` est une liste

6. Quelle condition teste la victoire du joueur ?

- A. `j == o`
- B. `(j,o)` dans la liste des paires gagnantes
- C. `o in j`
- D. `j in o`

7. Type de `score_j` après `score_j = 0` :

- A. `str`
- B. `int`
- C. `float`
- D. `bool`

8. Que fait `lower()` ?

- A. Met en majuscules
- B. Met en minuscules
- C. Enlève espaces
- D. Rien

9. Combien de fois s'exécute `for i in range(5):` ?

- A. 4
- B. 5
- C. 6
- D. 0

10. Quelle instruction ajoute 1 à `score_o` ?

- A. `score_o = score_o + 1`
- B. `score_o += 1`
- C. Les deux
- D. Aucune

11. Ligne correcte pour un choix au hasard :

- A. `random.pick(options)`
- B. `random.choice(options)`
- C. `choice.random(options)`
- D. `options.choice()`

12. Test d'égalité :

- A. `j = o`
- B. `j == o`
- C. `j != o`
- D. `(j,o) in ...`

13. Effet de `continue` dans une boucle :

- A. Arrête le programme
- B. Passe à l'itération suivante
- C. Recommence l'itération actuelle
- D. Recommence la boucle du début

14. Structure adaptée pour « jusqu'à 3 points » :

- A. `for i in range(3)`
- B. `while score_j < 3 and score_o < 3`
- C. `if score_j < 3`
- D. `match score_j:`

15. Test de validité de la saisie :

- A. `if options in j:`
- B. `if j in options:`
- C. `if j == options:`
- D. `if j not options:`

16. Effet de `print("Score :", a, "-", b)` :

- A. Concatène sans espaces
- B. Affiche avec espaces entre les éléments
- C. Additionne les variables
- D. Convertit en chaîne sans afficher

17. Règles correctes P-F-C :

- A. Pierre > Feuille, Feuille > Ciseaux, Ciseaux > Pierre
- B. Pierre > Ciseaux, Feuille > Pierre, Ciseaux > Feuille
- C. Pierre > Ciseaux, Feuille > Ciseaux, Ciseaux > Pierre
- D. Pierre > Feuille, Feuille > Pierre, Ciseaux > Ciseaux

18. Effet de `input(...).strip().lower()` :

- A. Rien
- B. Supprime espaces autour et met en minuscules
- C. Transforme en entier
- D. Mélange les lettres

19. Valeurs de `range(5)` :

- A. [1,2,3,4,5]
- B. [0,1,2,3,4]
- C. 5
- D. (0..5)

20. Où compter les égalités ?

- A. Dans le bloc `if j == o`
- B. Dans le test de victoire
- C. Après l'affichage final
- D. Avant la boucle